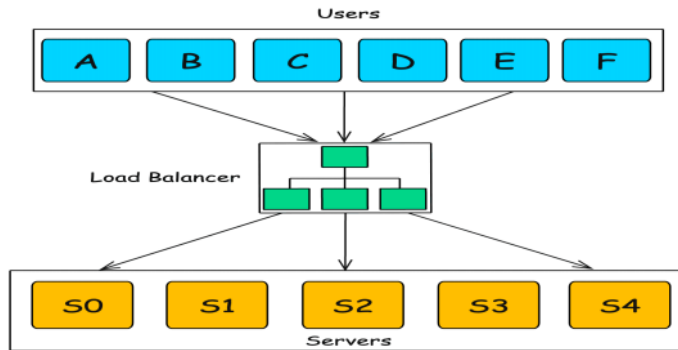


The Problem with Traditional Hashing

Before diving into consistent hashing, let us understand why the naive approach fails so badly.

Imagine you're building a **high-traffic web application** that serves millions of users daily. To handle the load efficiently, you distribute incoming requests across multiple backend servers using a **hash-based load balancer**.

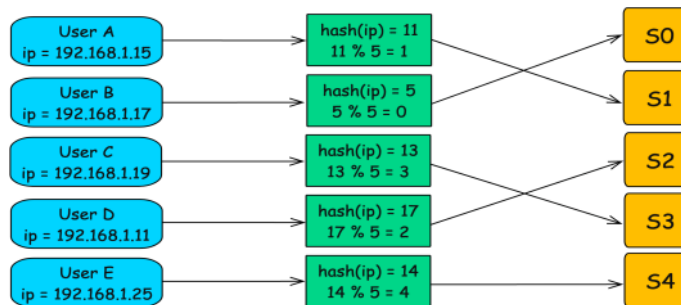
Your system consists of **5 backend servers** (S_0, S_1, S_2, S_3, S_4), and requests are assigned using a hash function that maps each user's **IP address to a specific server**.



The process works like this:

1. The load balancer takes a user's IP address (or session ID).
2. A **hash function** maps the IP to one of the backend servers by taking the **sum of bytes in the IP address** and computing **mod 5** (since we have 5 servers).
3. The request is **routed to the assigned server**, ensuring that the same user is always directed to the same server for session consistency.

Example:

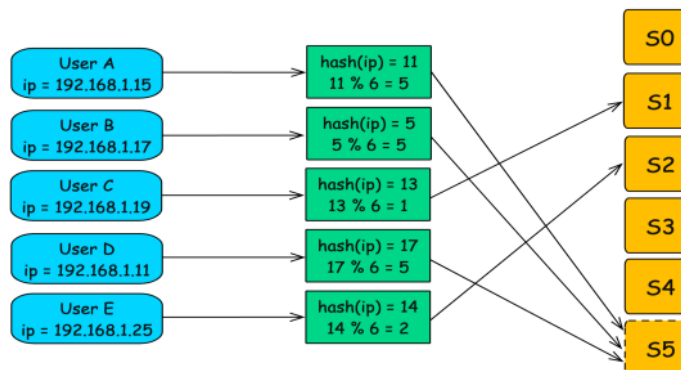


Everything Works Fine... Until You Scale

This approach works **as long as the number of servers remains constant**. But what happens when you **add or remove a server**?

Scenario 1: Adding a New Server (S5)

As traffic increases, you decide to **scale up** by adding a new backend server (S_5). Now, the hash function must be modified to use **mod 6** instead of **mod 5** since we have 6 servers now.

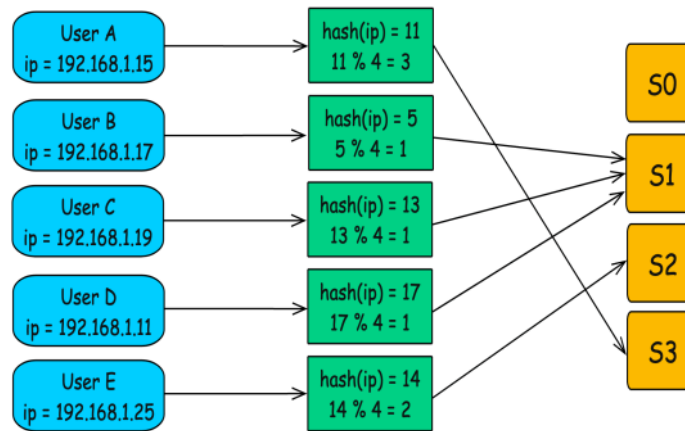


This seemingly simple change **completely disrupts the existing mapping**, causing **most users to be reassigned to different servers**.

This results into **massive rehashing**, leading to **high overhead**, and **potential downtime**.

Scenario 2: Removing a Server (S4)

Now, let's say one of the servers (S_4) fails or is removed. The number of servers drops to 4, forcing the hash function to switch from **mod 5** to **mod 4**.



Even though only **one** server was removed, **most users are reassigned to different servers**. This can cause:

- **Session Loss:** Active users may be logged out or disconnected.
- **Cache invalidation:** Cached data becomes irrelevant, increasing database load.
- **Severe performance degradation:** The system may struggle to run efficiently.

The Solution: Consistent Hashing

Consistent hashing offers a more scalable and efficient solution by ensuring that only a **small fraction of users** are reassigned when scaling up or down.

It performs really well when operated in dynamic environments, where the system scales up and down frequently.

How Consistent Hashing Works

Consistent hashing is a **distributed hashing technique** used to efficiently distribute data across multiple nodes (servers, caches, etc.).

It uses a **circular hash space** (hash ring) with a large and constant hash space.

Both nodes (servers, caches, or databases) and keys (data items) are mapped to positions on this hash ring using a **hash function**. Unlike modulo-based hashing, where changes in the number of nodes cause large-scale remapping, consistent hashing ensures that only a small fraction of keys are reassigned when a node is added or removed, making it highly scalable and efficient.

In consistent hashing, when the number of nodes changes, only k/n keys need to be reassigned, where k is the total number of keys and n is the total number of nodes.

2.1 Constructing the Hash Ring

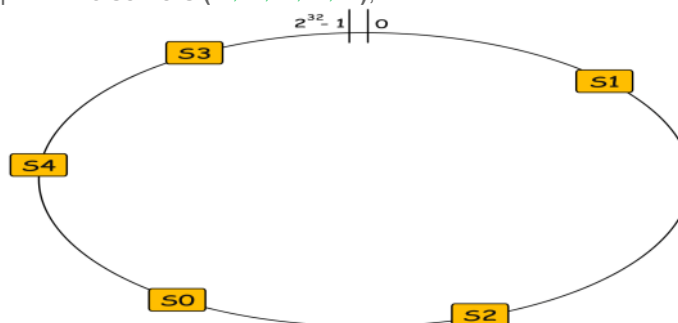
Instead of distributing keys based on $\text{Hash}(\text{key}) \bmod N$, consistent hashing places both servers and keys on a circular hash ring.

Defining the Hash Space

- We use a large, fixed hash space ranging from 0 to $2^{32} - 1$ (assuming a 32-bit hash function).
- This creates a circular structure, where values wrap around after reaching the maximum limit.

Placing Servers on the Ring

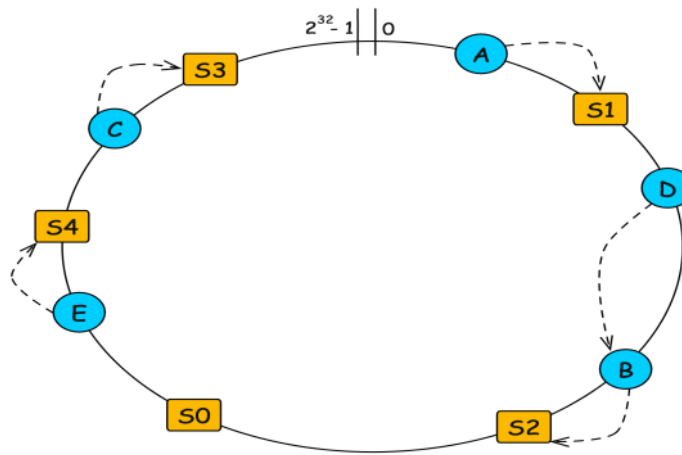
- Each **server (node)** is assigned a position on the hash ring by computing $\text{Hash}(\text{server_id})$.
- Using the above example with **5 servers** (S0, S1, S2, S3, S4), the hash function distributes them at different positions around the ring.



Mapping Keys to Servers

- When a key is added, its position is determined by computing $\text{Hash}(\text{key})$.
 - Example: a user's request is assigned a position on the ring based on the hash of their IP address: $\text{Hash}(\text{IP Address})$
- We then move clockwise around the ring until we find the next available server.
- The key (or request) is assigned to this server for storage or retrieval.

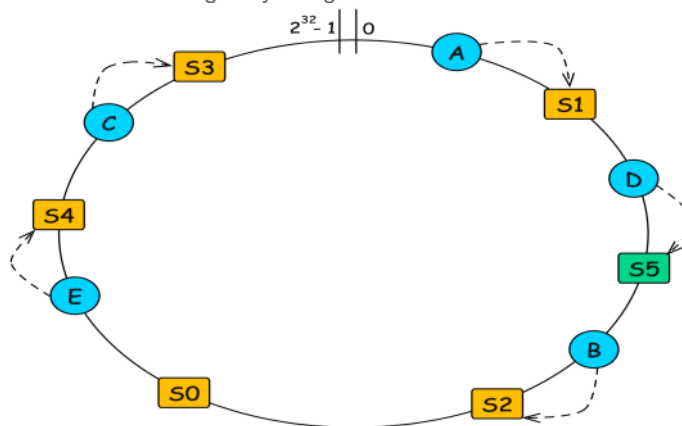
Note: In case a key's hash falls directly on a node's position, it belongs to that node.



2.2 Adding a New Server

Suppose we add a **new server (S5)** to the system.

- The position of **S5** falls between **S1** and **S2** in the hash ring.
- S5** takes over all keys (requests) that fall between **S1** and **S5**, which were previously handled by **S2**.
- Example:** User **D**'s requests which were originally assigned to **S2**, will now be redirected to **S5**.

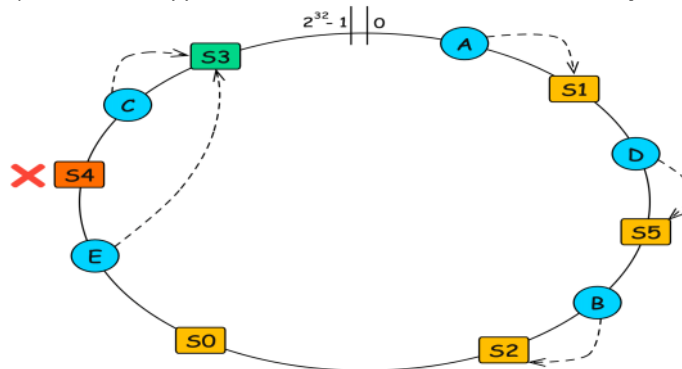


This demonstrates how consistent hashing efficiently redistributes keys with minimal disruption, ensuring that only a small subset of keys are reassigned when new servers are added.

2.3 Removing a Node

When a server, such as **S4**, fails or is removed from the system:

- All keys previously assigned to **S4** are reassigned to the next available server in the ring (**S3**).
- Only the keys (requests) that were mapped to **S4** need to move, while all other keys remain unaffected.



This results in **minimal data movement**, unlike traditional hashing where removing a node would require reassigning most keys.

3. Virtual Nodes

In **basic consistent hashing**, each server is assigned a **single position** on the hash ring. However, this can lead to **uneven data distribution**, especially when:

- The number of servers is small.
- Some servers accidentally get clustered together, creating **hot spots**.
- A server is **removed**, causing a sudden load shift to its immediate neighbor.

Virtual nodes (VNodes) are a technique used in consistent hashing to improve **load balancing** and **fault tolerance** by distributing data more evenly across servers.

How Virtual Nodes Work

Instead of assigning one position per server, each physical server is assigned **multiple positions** (virtual nodes) on the hash ring.

- Each server is hashed multiple times to different locations on the ring.
- When a request (key) is hashed, it is assigned to the next virtual node in a clockwise direction.

- The request is then routed to the actual server associated with the virtual node.

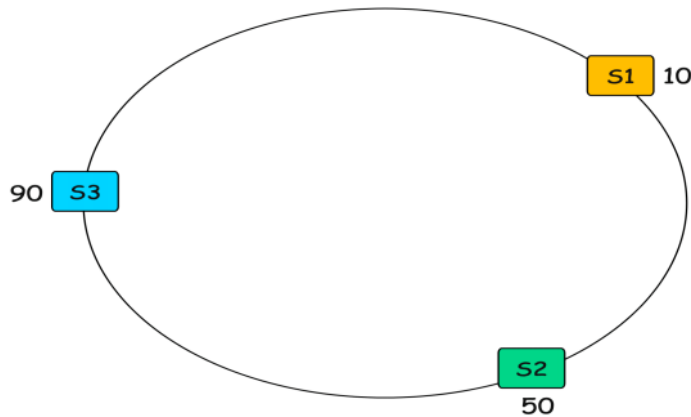
Example

Assume we have **three physical servers (S1, S2, S3)**. Without virtual nodes, their positions might be:

S1 → Position 10

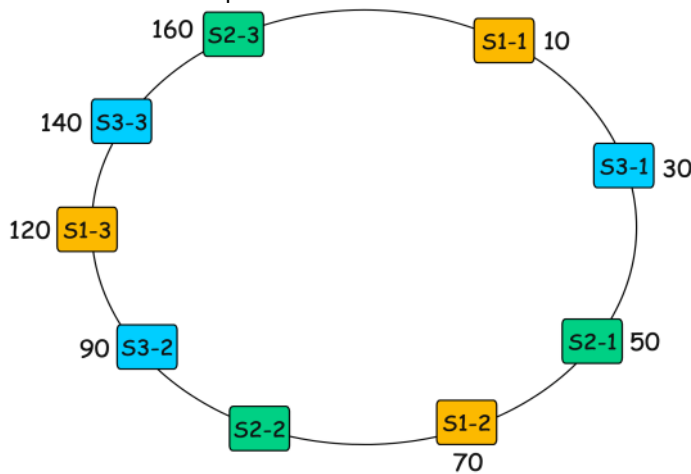
S2 → Position 50

S3 → Position 90



If **S1 fails**, all its keys must be reassigned to **S2**, which can create an overload.

With **virtual nodes**, each server is hashed multiple times:



Now, instead of just one point, **S1** is represented at **multiple positions**, making the distribution **more even**.

If **S1 fails**, its keys are more evenly redistributed among **S2 and S3**, rather than all going to **S2**.

CODE:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Security.Cryptography;
using System.Text;
public class CS
{
    // Number of virtual nodes for each physical server
    private readonly int numReplicas;
    // Sorted dictionary represents the hash ring
    // Key = Hash value
    // Value = Physical server name
    private readonly SortedDictionary<long, string> ring;
    // Stores all physical servers
    private readonly HashSet<string> servers;
    // Constructor
    public CS(List<string> serverList, int numReplicas)
    {
        this.numReplicas = numReplicas;
        this.ring = new SortedDictionary<long, string>();
        this.servers = new HashSet<string>();
        // Add all servers to the ring
        foreach (string server in serverList)
        {
            AddServer(server);
        }
    }
    // Generates a hash value using MD5
    private long Hash(string key)
    {
        using (MD5 md5 = MD5.Create())
        {
            byte[] inputBytes = Encoding.UTF8.GetBytes(key);
            byte[] digest = md5.ComputeHash(inputBytes);
        }
    }
}
```

```

        // Use first 4 bytes to create a 32-bit positive number
        long hash =
            ((long)(digest[0] & 0xFF) << 24) |
            ((long)(digest[1] & 0xFF) << 16) |
            ((long)(digest[2] & 0xFF) << 8) |
            ((long)(digest[3] & 0xFF));
        return hash;
    }
}

public void AddServer(string server)
{
    // Prevent duplicate addition
    if (!servers.Add(server))
        return;
    // Create virtual nodes
    for (int i = 0; i < numReplicas; i++)
    {
        long hash = Hash(server + "-" + i);
        ring[hash] = server;
    }
}

public void RemoveServer(string server)
{
    // Prevent removal of non-existent server
    if (!servers.Remove(server))
        return;
    // Remove virtual nodes
    for (int i = 0; i < numReplicas; i++)
    {
        long hash = Hash(server + "-" + i);
        ring.Remove(hash);
    }
}

// Find which server should handle the given key
public string GetServer(string key)
{
    // No servers available
    if (ring.Count == 0)
        return null;
    long hash = Hash(key);
    // Find the first hash >= key hash
    foreach (var entry in ring)
    {
        if (entry.Key >= hash)
            return entry.Value;
    }
    // Wrap around to the first server
    return ring.First().Value;
}

}

public class ConsistentHashing
{
    public static void Main(String[] args)
    {
        List<string> servers = new List<string>
        {
            "S0", "S1", "S2", "S3", "S4", "S5"
        };
        // Each server has 3 virtual nodes
        CS ch = new CS(servers, 3);
        // Assign keys to servers
        Console.WriteLine("UserA is assigned to: " + ch.GetServer("UserA"));
        Console.WriteLine("UserB is assigned to: " + ch.GetServer("UserB"));
        // Add a new server dynamically
        ch.AddServer("S6");
        Console.WriteLine("UserA is now assigned to: " + ch.GetServer("UserA"));
        // Remove a server dynamically
        ch.RemoveServer("S2");
        Console.WriteLine("UserB is now assigned to: " + ch.GetServer("UserB"));
    }
}

```